

SESSION 1 · BUILD YOUR OWN CLAW

Meet Your Claw: A Harness in Three Lines of C#

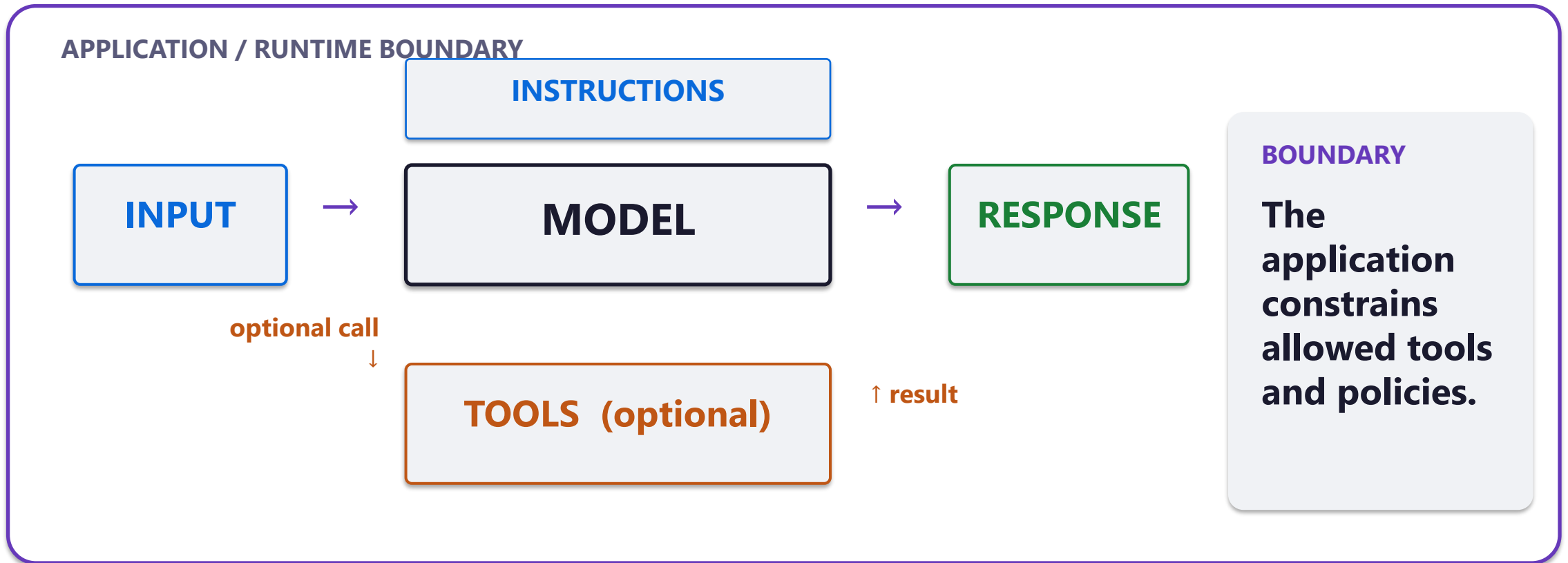
One finance assistant. An incremental capability ladder.

aka.ms/mafclaw/1

MOCK MARKET DATA · EDUCATIONAL · NOT FINANCIAL ADVICE

WHAT IS AN AGENT?

A model-guided loop inside an application boundary



Tools and context are optional — not part of every agent definition.

Start with an agent

```
using Azure.AI.Projects;
using Azure.Identity;
using Microsoft.Agents.AI;
using Microsoft.Extensions.Configuration;

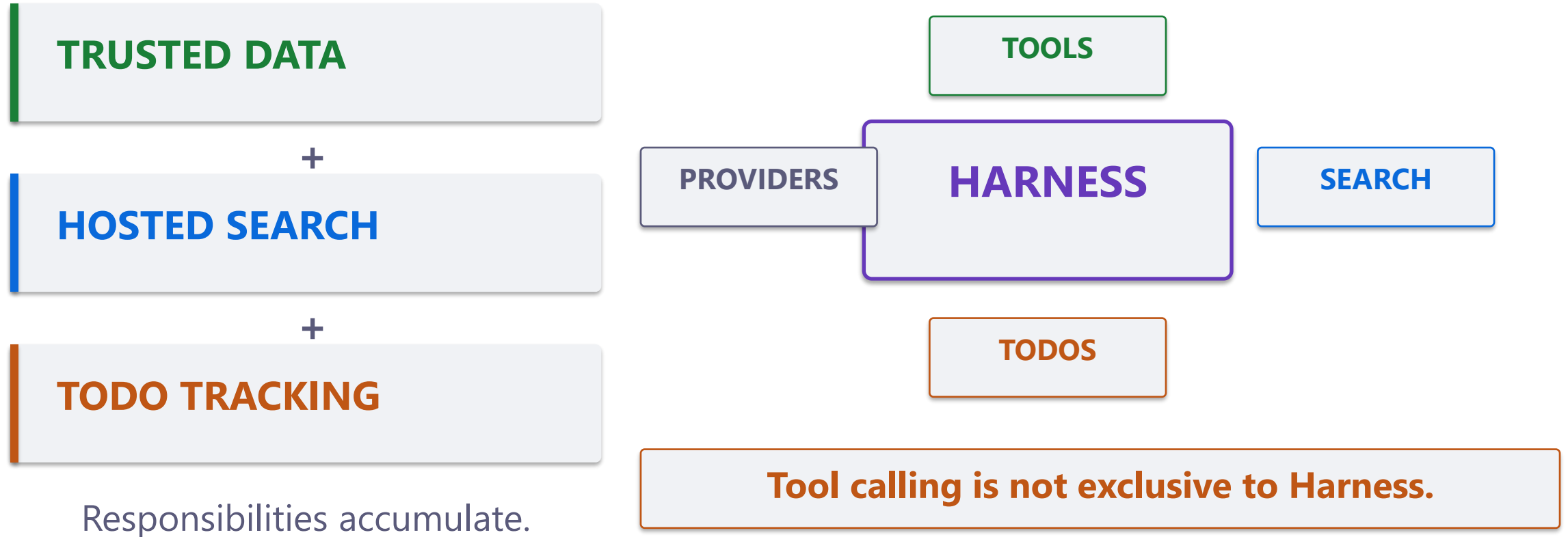
var config = new ConfigurationBuilder()
    .AddUserSecrets<Program>().AddEnvironmentVariables().Build();
var endpoint = config["Foundry:ProjectEndpoint"]!;
var model = config["Foundry:Model"] ?? "gpt-5-mini";

AIAgent agent = new AIProjectClient(new Uri(endpoint), new AzureCliCredential())
    .AsAIAgent(
        model: model,
        instructions: "You are a personal finance ...",
        name: "mafclaw-hello");

Console.WriteLine(await agent.RunAsync(
    "What are three things a beginner should consider before buying a stock?"));
```

✓ Already an agent

Why a composition layer?



Responsibilities accumulate.

Baseline $\neq$ incapable. Harness = composition standard.

Same config, different construction

BEFORE · checkpoint 01 · (...) = same constructor args · instruction text identical

```
AIAgent agent = new AIProjectClient(...)
    .AsAIAgent(
        model: model,
        instructions: "You are a personal finance ...",
        name: "mafclaw-hello");
```

AFTER · checkpoint 02 · +2 usings: Azure.AI.Extensions.OpenAI, Microsoft.Extensions.AI

```
IChatClient chatClient = new AIProjectClient(...)
    .GetProjectOpenAIClient()
    .GetResponsesClient()
    .AsIChatClient(model);

AIAgent agent = chatClient.AsHarnessAgent(new HarnessAgentOptions
{
    DisableFileMemory = true,
    ChatOptions = new ChatOptions
    {
        Instructions = "You are a personal finance ..."
    }
});
```

Capabilities, live



**TRUSTED
DATA**

**HOSTED
SEARCH**

**TODO
TRACKING**

Watch the terminal.

Who owns what

Clear boundaries keep the story honest.

AGENT FRAMEWORK

- AI Agent + sessions
- Runs + tool calling
- Reusable context providers

HARNESS

- Composition conventions
- Hosted capability wiring
- Provides TodoProvider

CONSOLE

- Interactive REPL loop
- /todos — inspect tracked work
- /exit — clean shutdown

Same scenario. More deliberate loop.

BEFORE

- Plain Agent Framework agent
- AsAIAgent convenience API
- Same instructions + prompt



AFTER

- Explicit Responses IChatClient
- Deterministic finance tool
- Hosted search + citations
- Todo tracking via Harness

This session

aka.ms/mafclaw/1

Series

aka.ms/mafclaw

Blog

aka.ms/mafclaw/blog

Repo

aka.ms/mafclaw/repo

Official post

devblogs.microsoft.com/agent-framework/meet-your-agent-harness-and-claw/

Rules, agents, and expanding scope

DETERMINISTIC RULES

Stable, testable logic

Explicit validation and calculations

+

BOUNDED AGENT STEP

Interprets variable requests

Selects among allowed tools

1 GENERATED RESPONSE

The model returns useful text.



2 TOOL-ASSISTED STEP

The model may select an allowed tool.



3 BOUNDED WORKFLOW

Application code sequences and enforces stops.

Code enforces boundaries; agents handle interpretation. Stop at the smallest scope that meets the need.